

Web программирование

Курс для бакалавриата

Чернышев Вячеслав

Взаимодействие с базами данных

Coding samples

<https://github.com/itmo-webdev/webdev-2025>

<https://github.com/slaveeks/webdev>



Содержание лекции

1. FastAPI: Dependency Injection
2. Databases: Postgres
3. SQLAlchemy
4. Alembic
5. Databases: MongoDB
6. Building large applications

SOLID

SOLID – один из самых популярных наборов принципов проектирования в разработке объектно-ориентированного программного обеспечения.

Аббревиатура для пяти принципов проектирования:

- **S**ingle Responsibility Principle
- **O**pen/Closed Principle
- **L**iskov Substitution Principle
- **I**nterface Segregation Principle
- **D**ependency Inversion

Dependency inversion (инверсия зависимостей)

- Модули высокого уровня не должны зависеть от модулей низкого уровня. Оба типа должны зависеть от абстракций
- Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций

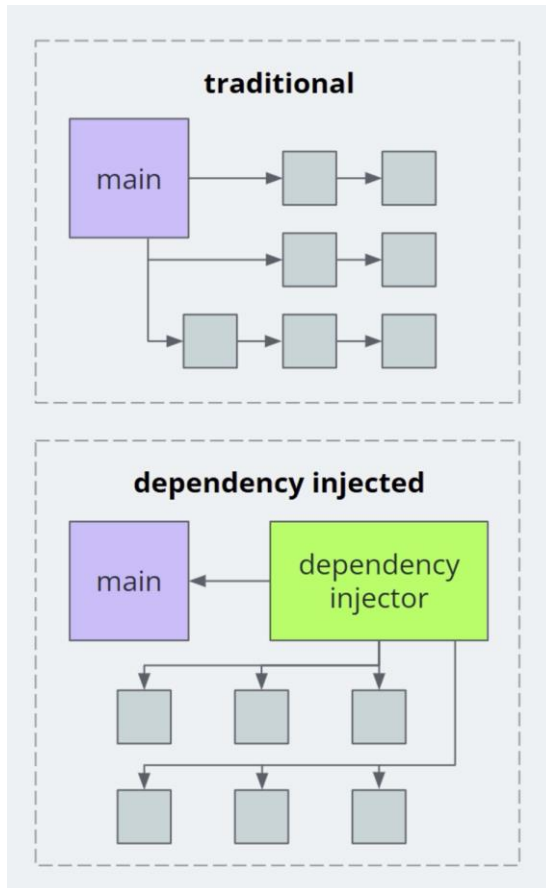
Паттерн **Dependency injection** (внедрение зависимостей) позволяет соблюдать второй пункт

Dependency injection

Паттерн, при котором зависимости объекта не создаются внутри него, а передаются извне в неинициализированном виде

Основные элементы:

- Контейнер, в котором хранятся инициализированные зависимости
- Способ получения зависимости из контейнера (инъекция)



Прямые зависимости vs абстракции



Плохо

```
class Email:
    def send(self, message):
        print(f"Sending email: {message}")

class Sms:
    def send(self, message):
        print(f"Sending Sms: {message}")

class Notification:
    def __init__(self):
        self.email = Email()
        self.sms = Sms()

    def notify_email(self, message):
        self.email.send(message)

    def notify_sms(self, message):
        self.sms.send(message)

notification = Notification()
notification.notify_email("Hello, world!")
notification.notify_sms("Hello, world!")
```



Хорошо

```
class Email:
    def send(self, message):
        print(f"Sending email: {message}")

class Sms:
    def send(self, message):
        print(f"Sending Sms: {message}")

class Notification:
    def __init__(self, method):
        self.method = method

    def notify(self, message):
        self.method.send(message)

notification = Notification(Email())
notification.notify("Hello, world!")

notification = Notification(Sms())
notification.notify("Hello, world!")
```


DI в FastAPI

```
@router.get("/items")
async def get_items(db: DatabaseProto = Depends(database)):
    return await db.get_all()
```

1. Зависимость от абстракции DatabaseProto
2. Используется класс Depends или наследники (Header, Body, Query)
3. database это callable объект, который возвращает экземпляр БД (может быть синхронным и асинхронным)

DI в FastAPI

Реализация работы с БД

```
class Dict(DatabaseProto):  
    def __init__(self):  
        self.items = [i for i in range(10)]  
  
    async def get_all(self):  
        return self.items  
  
    async def get_session(self):  
        return DictSession(d=self)
```

Callable объект

```
# Async dependency  
async def database() -> DatabaseProto:  
    return Dict()  
  
# Sync dependency  
def database() -> DatabaseProto:  
    return Dict()
```

DI в FastAPI

Callable класс

```
class Database:
    def __init__(self):
        db = Dict()
        orig_get_all = db.get_all

        async def track():
            t = time()
            try:
                return await orig_get_all()
            finally:
                print(f"Time: {time() - t}")
        self.db = db
        self.db.get_all = track

    def __call__(self):
        return self.db
```

```
@router.get("/items")
async def get_items(db: DatabaseProto = Depends(Database())):
    return await db.get_all()
```

Рекурсивные зависимости

item-session route -> session -> Database

```
async def session(db: DatabaseProto = Depends(Database())) -> SessionProto:
    s = await db.get_session()
    return s

@router1.get("/items-session")
async def get_items_session(session: SessionProto = Depends(session)):
    return await session.execute()
```

Контекстные менеджеры

Позволяют корректно открывать и закрывать соединения, делать код надежнее и понятнее

```
with open("example.txt", "w") as f:
    f.write("Привет, мир!")
# файл автоматически закроется
```

```
@contextmanager
def timer(name="Блок кода"):
    start = time.time()
    yield
    end = time.time()
    print(f"{name} занял {end - start:.4f} секунд")

with timer("Обработка"):
    sum(range(10**6))
```

```
class ManagedResource:
    def __enter__(self):
        print("Ресурс открыт")
        return "ресурс"

    def __exit__(self, exc_type, exc_value, traceback):
        print("Ресурс закрыт")

with ManagedResource() as res:
    print(f"Используем {res}")
```

Контекстный менеджер fastapi

Используется ключевое слово yield

```
async def session(db: DatabaseProto = Depends(Database())) -> SessionProto:
    s = await db.get_session()
    try:
        yield s
    finally:
        await s.close()

@router1.get("/items-session")
async def get_items_session(session: SessionProto = Depends(session)):
    return await session.execute()
```

```
Execute session
Time: 1.1920928955078125e-06
Close session
INFO: 127.0.0.1:58648 - "GET /items-session HTTP/1.1" 200 OK
```

Зависимости и поведение приложения

```
async def verify_token(token: str = Header()) -> bool:
    if token != "123":
        raise HTTPException(status_code=401)
    return True
```

```
@router.get("/items")
async def get_items(
    db: DatabaseProto = Depends(Database()),
    is_valid: bool = Depends(verify_token)
):
    print(is_valid)
    return await db.get_all()
```

default

GET /items Get Items

Parameters

Name	Description
token * required string (header)	token

Execute

Зависимости и поведение приложения

На уровне роутера

```
app = FastAPI()  
router = APIRouter(dependencies=[Depends(verify_token)])
```

На уровне приложения

```
app = FastAPI(dependencies=[Depends(verify_token)])  
router = APIRouter()
```

Для использования в ручках можно дублировать, результат кэшируется по умолчанию

Реализация DI в fastapi. Итого

1. Передаем реализацию в конкретные вьюшки
2. Объявляем зависимости, которые могут зависеть друг от друга
3. Осуществляем операции до и после запроса через зависимости, как в контекстных менеджерах
4. Упрощаем и повышаем безопасность кода, устанавливая зависимости на нужных уровнях

Реализация DI в fastapi. Плюсы и минусы



1. Наглядность
2. Swagger
3. Простота



1. Только для view-функций (нельзя для класса или функции)
2. Нет контейнера (нет единого объекта с зависимостями)
3. Копипасты callable-объектов

Недостатки решения fastapi

1. Так как мы используем функцию **service** не всегда мы не можем прокинуть в нее **Depends()**
2. Db приходится передавать передавать все далее и далее, хотя **service** не зависит от него
3. Если репозитории много, то приходится каждый раз дублировать инициализацию

```
async def multiple_service(db: DatabaseProto):
    res = []
    data = await db.get_all()
    for i in data:
        res.append(i * 2)
    return res

async def multiply_by_api(data: list):
    res = []
    for i in data:
        res.append(i * 3)
    return res

async def service(db: DatabaseProto):
    data = await multiple_service(db)
    return await multiply_by_api(data)

@router.get("/items")
async def get_items(
    db: DatabaseProto = Depends(Database()),
    cache: CacheProto = Depends(cache)
):
    cached_items = cache.get("items")
    if cached_items is None:
        data = await service(db)
        return data
    return cached_items
```

Вариант правильной реализации

```
class Container(containers.DeclarativeContainer):  
    database = providers.Factory(Dict)  
    cache = providers.Factory(ObjectCache)
```

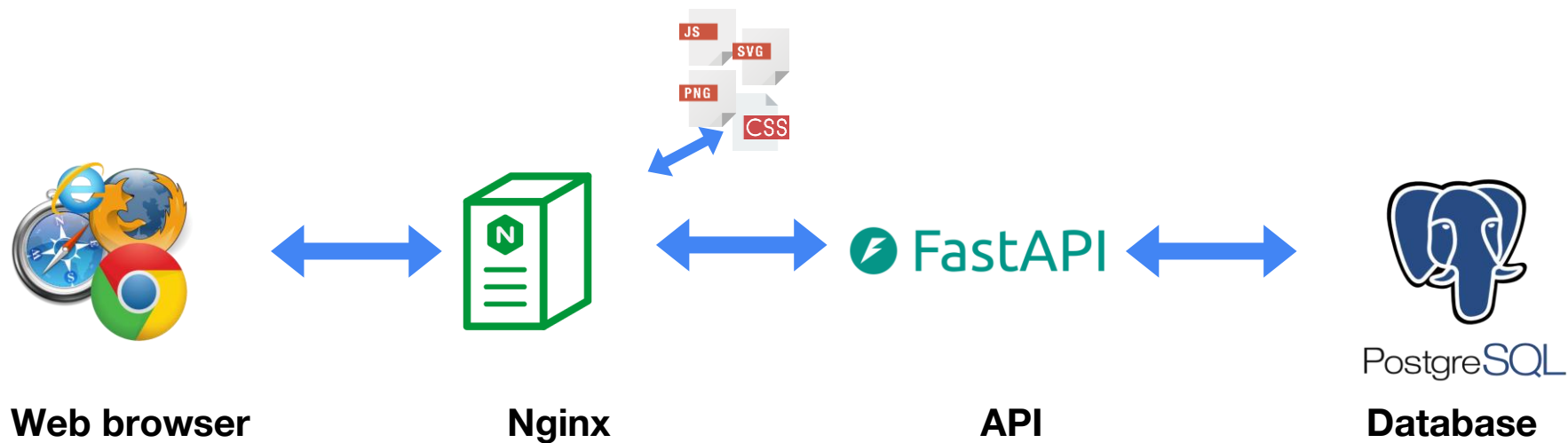
```
async def multiple_service(database: DatabaseProto): ...  
  
async def multiply_by_api(data: list): ...  
  
async def service():  
    data = await multiple_service()  
    return await multiply_by_api(data)  
  
@router.get("/items")  
@inject  
async def get_items(  
    database: DatabaseProto = Depends(Provide[Container.database]),  
    cache: CacheProto = Depends(Provide[Container.cache])  
):  
    cached_items = cache.get("items")  
    if cached_items is None:  
        data = await service()  
        return data  
    return cached_items  
  
app.include_router(router)  
  
app.container = container  
container.wire(modules=[__name__])
```

<https://python-dependency-injector.ets-labs.org/examples/fastapi.html>

Databases



Архитектура Веб приложения с БД



Зачем нужны БД и какие их виды бывают?

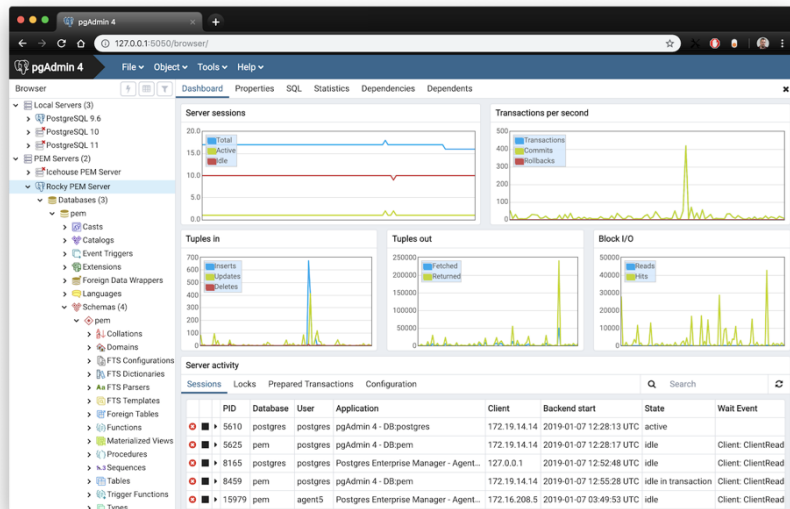
Хранить данные в плоском файле не эффективно. Базы данных организуют данные в правильном порядке, эффективном для чтения и записи.

Популярные типы СУБД:

1. Реляционные — [PostgreSQL](#), MySQL, [SQLite](#)
2. Key-value — [Redis](#)
3. Документно-ориентированные — [MongoDB](#)
4. Колоночные — ClickHouse
5. Time-series — Prometheus
6. Графовые — Neo4J

PostgreSQL

<https://www.pgadmin.org/>



<https://www.docker.com/>



database > `docker-compose.yml`

```
1  version: '3.8'
2
3  services:
4    postgres:
5      image: postgres:15
6      restart: unless-stopped
7      environment:
8        POSTGRES_DB: webdev
9        POSTGRES_USER: user
10       POSTGRES_PASSWORD: admin
11       PGDATA: /var/lib/postgresql/data/pgdata
12     ports:
13       - "5432:5432"
14     volumes:
15       - ./data:/var/lib/postgresql/data
```


Запуск postgres

```
> docker compose up -d
WARN[0000] /Users/slaveeks/Documents/itmo/webdev/lesson-2/database/docker-compose.yml: the attribute `version`
is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 15/15
✓ postgres Pulled
  9a04620f8454 Pull complete
  e70ba1e7b606 Pull complete
  5027f749dcef Pull complete
  b2feff975e6d Pull complete
  2f048a26e655 Pull complete
  1da1c7240a39 Pull complete
  582b924004c5 Pull complete
  2673eb93ed9e Pull complete
  016a80734744 Pull complete
  a7110de33e28 Pull complete
  60b2004d7ded Pull complete
  347920de26f3 Pull complete
  b61361295790 Pull complete
  2177ca3c5f7d Pull complete
[+] Running 2/2
✓ Network database_default Created
✓ Container database-postgres-1 Started
```

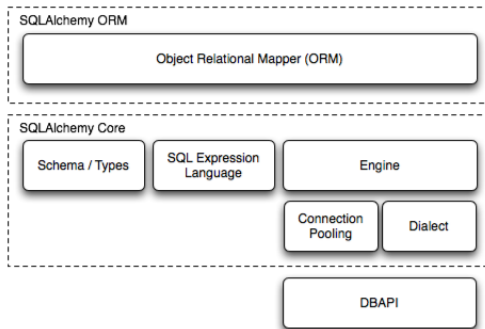
☐	☐	database	-	-	-	0%	3 minute	☐	:	
☐	☐	postgres-1	2d7c44154b2d	postgres:1!	5432:5432	0%	3 minute	☐	:	

SQLAlchemy

Библиотека для доступа к SQL базам данных

Особенности:

- ORM
- Сырые SQL запросы
- Модульность, расширяемость
- Поддержка транзакций



Database	Fully tested in CI	Normal support	Best effort
Microsoft SQL Server	2017	2012+	2005+
MySQL / MariaDB	5.6, 5.7, 8.0 / 10.8, 10.9	5.6+ / 10+	5.0.2+ / 5.0.2+
Oracle	18c	11+	9+
PostgreSQL	12, 13, 14, 15	9.6+	9+
SQLite	3.36.0	3.12+	3.7.16+

Устанавливаем SQLAlchemy

- Для синхронного кода `pip install sqlalchemy psycopg2_`
- Для асинхронного кода `pip install sqlalchemy[asyncio] asyncpg_`
- С драйвером PostgreSQL `pip install sqlalchemy[postgresql]`
`pip install sqlalchemy[postgresql-asyncpg]`

Синхронное подключение к базе данных

```
from sqlalchemy import create_engine
from sqlalchemy import text

engine = create_engine("postgresql://user:admin@localhost:5432/webdev")

with engine.connect() as conn:
    result = conn.execute(text("SELECT 'Hello World'"))
    print(result.all())

engine.dispose()
```

Асинхронное подключение

```
import asyncio
from sqlalchemy.ext.asyncio import create_async_engine
from sqlalchemy import text

async def main():
    engine = create_async_engine("postgresql+asyncpg://user:admin@localhost:5432/webdev")
    async with engine.begin() as conn:
        result = await conn.execute(text("SELECT 'Hello World'"))
        print(result.all())

asyncio.run(main())
```

Классическая модель

```
from sqlalchemy import MetaData, Table, Column, Integer, String

# Контейнер для моделей
metadata_obj = MetaData()

user_table = Table(
    "user",
    metadata_obj,
    Column("id", Integer, primary_key=True),
    Column("name", String(30), nullable=False),
    Column("fullname", String),
)
```

Типы колонок

- > Integer/BigInteger/SmallInteger
- > String
- > Boolean
- > Float/Double
- > Date
- > Datetime
- > Numeric
- > Enum
- > Text

Связи между таблицами

- One to many
- Many to one
- One to one
- Many to many

One-to-many

back_populates - имя
атрибута связанной модели

```
class Parent(Base):
    __tablename__ = "parent_table"

    id: Mapped[int] = mapped_column(primary_key=True)
    children: Mapped[List["Child"]] = relationship(back_populates="parent")

class Child(Base):
    __tablename__ = "child_table"

    id: Mapped[int] = mapped_column(primary_key=True)
    parent_id: Mapped[int] = mapped_column(ForeignKey("parent_table.id"))
    parent: Mapped["Parent"] = relationship(back_populates="children")
```

Many-to-one

```
class Parent(Base):
    __tablename__ = "parent_table"

    id: Mapped[int] = mapped_column(primary_key=True)
    child_id: Mapped[int] = mapped_column(ForeignKey("child_table.id"))
    child: Mapped["Child"] = relationship()

class Child(Base):
    __tablename__ = "child_table"

    id: Mapped[int] = mapped_column(primary_key=True)
```

One-to-one

```
class Parent(Base):
    __tablename__ = "parent_table"

    id: Mapped[int] = mapped_column(primary_key=True)
    child: Mapped["Child"] = relationship(back_populates="parent")

class Child(Base):
    __tablename__ = "child_table"

    id: Mapped[int] = mapped_column(primary_key=True)
    parent_id: Mapped[int] = mapped_column(ForeignKey("parent_table.id"))
    parent: Mapped["Parent"] = relationship(back_populates="child")
```

Many-to-many

Используется связующая
таблица

Параметр **secondary** со
ссылкой на связующую
таблицу обязателен

```
association_table = Table(
    "association_table",
    Base.metadata,
    Column("left_id", ForeignKey("left_table.id")),
    Column("right_id", ForeignKey("right_table.id")),
)

class Parent(Base):
    __tablename__ = "parent_table"

    id: Mapped[int] = mapped_column(primary_key=True)
    children: Mapped[List["Child"]] = relationship(secondary=association_table)

class Child(Base):
    __tablename__ = "child_table"

    id: Mapped[int] = mapped_column(primary_key=True)
```

Транзакции

это последовательность SQL-операций (INSERT, UPDATE, DELETE и т. д.), которые объединяются в одну логическую единицу работы с данными

1. **Atomicity (Атомарность)** – все операции либо выполняются полностью, либо откатываются.
 2. **Consistency (Согласованность)** – база всегда переходит из одного корректного состояния в другое.
 3. **Isolation (Изолированность)** – параллельные транзакции не мешают друг другу.
 4. **Durability (Надёжность)** – результат зафиксированной транзакции сохраняется даже при сбоях.
- BEGIN или START TRANSACTION — начало транзакции.
 - COMMIT — зафиксировать изменения (сохранить их в БД).
 - ROLLBACK — отменить транзакцию (откатить изменения).
 - SAVEPOINT — промежуточная точка сохранения внутри транзакции, чтобы можно было откатиться только до неё.

Индексы

это специальная структура данных (обычно дерево или хэш), которая ускоряет поиск строк в таблице по определённым столбцам.

- **B-Tree индекс (обычный)** - для поиска по диапазонам (=, <, >, BETWEEN, LIKE 'abc%')
- **Уникальный индекс (UNIQUE)**
- **Хэш-индекс** - по точному совпадению
- **Составной индекс** - для поиска по нескольким полям

Оптимизация запросов

EXPLAIN показывает план выполнения запроса — то, как оптимизатор базы данных собирается его выполнить

- какие таблицы будут использованы,
- в каком порядке,
- будут ли задействованы индексы,
- какой ожидается объём данных,
- какова примерная стоимость операции.

Сессия

Позволяет производить несколько операций над объектом, создавая транзакцию автоматически

```
# Сессия без автокоммита
engine = create_engine("postgresql+asyncpg://user:admin@localhost:5432/webdev")
Session = sessionmaker(engine)
with Session() as session:
    session.add(some_object)
    session.commit()

# Сессия с автокоммитом
engine = create_engine("postgresql+asyncpg://user:admin@localhost:5432/webdev").execution_options(isolation_level="AUTOCOMMIT")
Session = sessionmaker(engine)
with Session() as session:
    session.add(some_object)
```

Можно подписываться на события:

<https://docs.sqlalchemy.org/en/20/core/event.html>

Создание объекта модели

```
from sqlalchemy import MetaData
from sqlalchemy.orm import declarative_base

Base = declarative_base(Metadata())

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]

Session = sessionmaker()

with Session() as session:
    user = User(name="John")
    # Привязываем объект к сессии, теперь хранится в базе
    session.add(user)
    # Генерируем первичный ключ, отправляем в базу, но он пока не сохраняется
    session.flush()
    # Коммитим
    session.commit()
```

Получение объекта

```
with Session() as session:
    data = session.execute(select(User).order_by(User.id)).first() # => result.user.name, result.user.id
    # SELECT user.id, user.name FROM users ORDER BY user.id
```

```
with Session() as session:
    data = session.scalars(select(User).order_by(User.id)).first() # => result.name, result.id
```

```
with Session() as session:
    data = session.exec(select(User, Role).join(User.role)) # => result.User, result.Role
    # SELECT user.id, user.name, role.id, role.name FROM users JOIN roles ON users.role_id = roles.id
```

```
with Session() as session:
    data = session.execute(select(User.id, Role.name).join(User.role).order_by(User.name)).scalars()
    # => result.id, result.name
    # SELECT user.id, role.name FROM users JOIN roles ON users.role_id = roles.id ORDER BY user.name
```

Получение объекта по условию

```
select(User).filter(User.id == 1)
# SELECT user.id, user.name FROM users WHERE user.id = 1
```

```
select(User)
    .filter(User.id.in_([1, 2]))
    .filter(User.name.like("John%"))
# SELECT user.id, user.name FROM users WHERE user.id IN (1, 2) AND user.name LIKE 'John%'
```

```
from sqlalchemy import or_
select(User).filter(or_(User.id == 1, User.name like "John%"))
# SELECT user.id, user.name FROM users WHERE user.id = 1 OR user.name LIKE 'John%'
```

Соединение объектов моделей

```
select(User).join(User.role)
select(User).join(Role, Role.id == User.role_id)
# SELECT user.id, user.name, role.id, role.name FROM users JOIN roles ON users.role_id = roles.id
```

```
select(User).join(User.role).join(Department, Department.id == User.department_id)
# SELECT user.id, user.name, role.id, role.name, department.id, department.name
# FROM users JOIN roles ON users.role_id = roles.id JOIN departments ON users.department_id = departments.id
```

```
select(User).join(User.role.and_(Role.is_admin == True))
# SELECT user.id, user.name, role.id, role.name FROM users JOIN roles ON users.role_id = roles.id AND role.is_admin = TRUE
```

Aliased

```
role_alias_1 = aliased(Role)
role_alias_2 = aliased(Role)

select(User)
    .join(role_alias_1, User.role)
    .where(role_alias_1.is_admin == True)
    .join(role_alias_2, User.role)
    .where(role_alias_2.is_staff == True)

# SELECT user.id, user.name, role_alias_1.id, role_alias_1.name, role_alias_2.id, role_alias_2.name
# FROM users JOIN roles AS role_alias_1 ON users.role_id = role_alias_1.id AND role_alias_1.is_admin = TRUE
# JOIN roles AS role_alias_2 ON users.role_id = role_alias_2.id AND role_alias_2.is_staff = TRUE
```

Подзапросы

```
✓ sub_role = select(Role)
    .filter(Role.id == 1)
    .subquery()

select(User).join(sub_role, User.role_id == sub_role.id)
# SELECT user.id, user.name
# FROM users JOIN (SELECT roles.id, roles.name FROM roles WHERE roles.id = 1) AS anon_1
# ON users.role_id = anon_1.id
```

Вставка

```
from sqlalchemy import insert
session.execute(insert(User), [{"name": "John"}, {"name": "Jane"}])
# INSERT INTO users (name) VALUES (?, ?) [(John), (Jane)]
```

insert.returning(User) вернет сохраненные данные

```
session.execute(insert(User).values(role=root), [{"name": "John"}, {"name": "Jane"}])
# INSERT INTO users (name, role_id) VALUES (?, 1), (?, 1) [(John), (Jane)]
```

```
session.execute(insert(User),
[
    {"name": "John", role_id: select(Role.id).filter(Role.name == "admin")}
],
)
# INSERT INTO users (name, role_id) VALUES
#(?, (SELECT roles.id FROM roles WHERE roles.name = 'admin'))
#[("John")]
```

Обновление

```
from sqlalchemy import update
session.execute(update(User), {"id": 1, "name": "John"}, [{"id": 2, "name": "Jane"}])
# UPDATE users SET name = ? WHERE id = ? [('John', 1), ('Jane', 2)]
```

```
from sqlalchemy import bindparam
session.execute(update(User).where(User.name == bindparam("old_name")),
    [{"old_name": "John", "name": "John1"}, {"old_name": "Jane", "name": "Jane1"}])
# UPDATE users SET name = ? WHERE name = ? [('John1', 'John'), ('Jane1', 'Jane')]
```


Удаление

```
from sqlalchemy import delete
session.execute(delete(User).where(User.role_id == 1))
# DELETE FROM users WHERE role_id = 1

session.execute(delete(User).where(User.role_id == select(Role.id).filter(Role.name == "admin")))
# DELETE FROM users WHERE role_id = (SELECT roles.id FROM roles WHERE roles.name = 'admin')
```

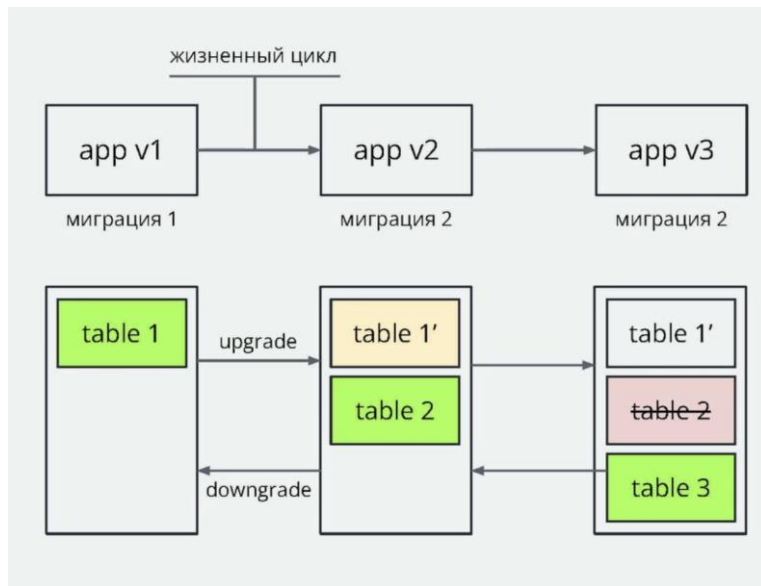
Сырые запросы

```
from sqlalchemy import text
session.execute(text("UPDATE users SET name = :name WHERE id = :id"), {"name": "John", "id": 1})
# UPDATE users SET name = 'John' WHERE id = 1

session.execute(text("SELECT * FROM users WHERE role_id = 2")).fetchall()
# SELECT * FROM users WHERE role_id = 2
```

Миграция

Код, который позволяет переводить
базу данных из одного состояния в
другое



Свойства миграций

- Транзакционность
- Версионированность
- Реверсивность

Инструменты:

- Файлы с SQL-запросами
- Yoyo-Migrations
- Django Migrations
- Alembic

Alembic

Это инструмент для управления миграциями БД в Python-проектах, разработанный специально для работы с SQLAlchemy



Преимущества:

- Автогенерация миграций
- Отслеживает и управляет версионированием
- Поддерживает автоматические и ручные миграции
- Широкая интеграция
- Удобный CLI

Установка Alembic

```
> pip install alembic psycopg2
Collecting alembic
  Downloading alembic-1.16.5-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: psycopg2 in /Users/slaveeks/Documents/itmo/webdev/lesson-2/venv/lib/python3.13/site-packages (2.9.10)
Requirement already satisfied: SQLAlchemy>=1.4.0 in /Users/slaveeks/Documents/itmo/webdev/lesson-2/venv/lib/python3.13/site-packages (from alembic) (2.0.43)
Collecting Mako (from alembic)
  Downloading mako-1.3.10-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: typing-extensions>=4.12 in /Users/slaveeks/Documents/itmo/webdev/lesson-2/venv/lib/python3.13/site-packages (from alembic) (4.15.0)
Collecting MarkupSafe>=0.9.2 (from Mako->alembic)
  Using cached MarkupSafe-3.0.2-cp313-cp313-macosx_11_0_arm64.whl.metadata (4.0 kB)
Downloading alembic-1.16.5-py3-none-any.whl (247 kB)
Downloading mako-1.3.10-py3-none-any.whl (78 kB)
Using cached MarkupSafe-3.0.2-cp313-cp313-macosx_11_0_arm64.whl (12 kB)
Installing collected packages: MarkupSafe, Mako, alembic
Successfully installed Mako-1.3.10 MarkupSafe-3.0.2 alembic-1.16.5
```

```
> alembic --version
alembic 1.16.5
```

Установка Alembic

```
> pip install alembic psycopg2
Collecting alembic
  Downloading alembic-1.16.5-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: psycopg2 in /Users/slaveeks/Documents/itmo/webdev/lesson-2/venv/lib/python3.13/site-packages (2.9.10)
Requirement already satisfied: SQLAlchemy>=1.4.0 in /Users/slaveeks/Documents/itmo/webdev/lesson-2/venv/lib/python3.13/site-packages (from alembic) (2.0.43)
Collecting Mako (from alembic)
  Downloading mako-1.3.10-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: typing-extensions>=4.12 in /Users/slaveeks/Documents/itmo/webdev/lesson-2/venv/lib/python3.13/site-packages (from alembic) (4.15.0)
Collecting MarkupSafe>=0.9.2 (from Mako->alembic)
  Using cached MarkupSafe-3.0.2-cp313-cp313-macosx_11_0_arm64.whl.metadata (4.0 kB)
Downloading alembic-1.16.5-py3-none-any.whl (247 kB)
Downloading mako-1.3.10-py3-none-any.whl (78 kB)
Using cached MarkupSafe-3.0.2-cp313-cp313-macosx_11_0_arm64.whl (12 kB)
Installing collected packages: MarkupSafe, Mako, alembic
Successfully installed MarkupSafe-3.0.2 Mako-1.3.10 alembic-1.16.5
```

```
> alembic --version
alembic 1.16.5
```

Инициализация

```
> alembic init alembic
```

▼ alembic

> versions

🐍 env.py

📖 README

≡ script.py.mako

≡ alembic.ini

- **alembic.ini** - главный конфиг файл (параметры подключения к БД, логирование)
- **env.py** - конфиг файл для работы с миграциями, работа с моделями
- **script.py.mako** - шаблон для генерации новых файлов миграций
- **versions/** - все созданные миграции

Alembic.ini

sqlalchemy.url - подключение к БД

script_location - путь до файлов

миграции и скриптов Alembic

file_template - как будут называться

файлы миграции

timezone - часовой пояс для

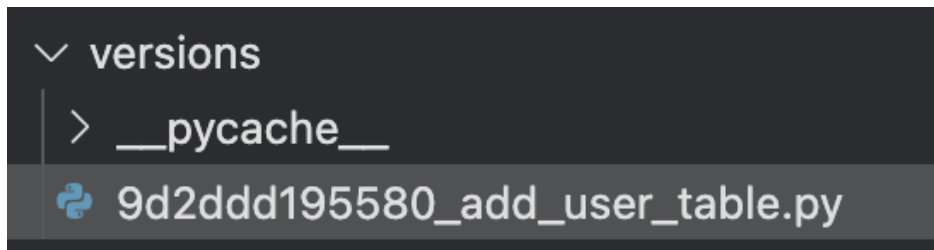
миграций

```
database > alembic > ≡ alembic.ini
1  # A generic, single database configuration.
2
3  [alembic]
4  # path to migration scripts.
5  # this is typically a path given in POSIX (e.g. forward slashes)
6  # format, relative to the token %(here)s which refers to the location of this
7  # ini file
8  script_location = %(here)s/alembic
9
10 # template used to generate migration file names; The default value is %(rev)s_%(slug)s
11 # Uncomment the line below if you want the files to be prepended with date and time
12 # see https://alembic.sqlalchemy.org/en/latest/tutorial.html#editing-the-ini-file
13 # for all available tokens
14 # file_template = %(year)d_%(month).2d_%(day).2d_%(hour).2d_%(minute).2d_%(rev)s_%(slug)s
15
16 # sys.path path, will be prepended to sys.path if present.
17 # defaults to the current working directory.  for multiple paths, the path separator
18 # is defined by "path_separator" below.
19 prepend_sys_path = .
20
```

```
# database URL.  This is consumed by the user-maintained env.py script only.
# other means of configuring database URLs may be customized within the env.py
# file.
sqlalchemy.url = postgresql://user:admin@localhost:5432/webdev
```

Ручной режим миграций

alembic revision -m "add user table"



revision - id миграции

down_revision - id пред. Миграции

upgrade() - для внесения изменений

downgrade() - для отката до пред. версии

```
"""add user table

Revision ID: 9d2ddd195580
Revises:
Create Date: 2025-09-19 09:47:21.794145

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa

# revision identifiers, used by Alembic.
revision: str = '9d2ddd195580'
down_revision: Union[str, Sequence[str], None] = None
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None

def upgrade() -> None:
    """Upgrade schema."""
    pass

def downgrade() -> None:
    """Downgrade schema."""
    pass
```

Ручная миграция

```
def upgrade() -> None:
    op.create_table('users',
        sa.Column('id', sa.Integer, primary_key=True, autoincrement=True),
        sa.Column('name', sa.String(100), nullable=False),
        sa.Column('email', sa.String(100), nullable=False, unique=True),
        sa.Column('created_at', sa.TIMESTAMP, nullable=False, server_default=sa.func.current_timestamp()),
    )

def downgrade() -> None:
    op.drop_table('users')
```

alembic upgrade head - применение всех миграций

alembic upgrade 9d2ddd195580 - применение миграций до определенной версии

- ▼  Tables (2)
 - >  alembic_version
 - >  users

Откатка миграций

alembic downgrade -1 - откат последней миграции

alembic downgrade 9d2ddd195580 - откат до конкретной версии

Автоматическая миграция

env.py

```
from app.models import Base

# this is the Alembic Config object, which
# access to the values within the .ini file
config = context.config

# Interpret the config file for Python logging
# This line sets up loggers basically.
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# add your model's MetaData object here
# for 'autogenerate' support
# from myapp import mymodel
# target_metadata = mymodel.Base.metadata
target_metadata = Base.metadata
```

app/models.py

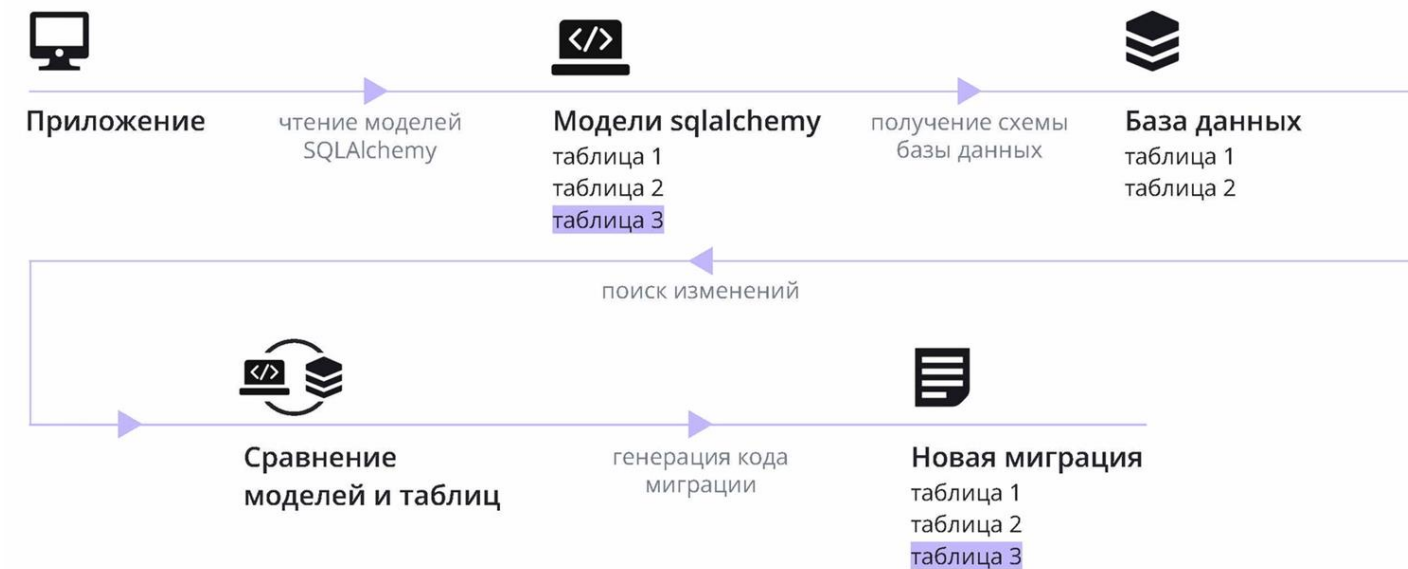
```
class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100), nullable=False)
    email: Mapped[str] = mapped_column(String(100), nullable=False, unique=True)
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())
```

alembic revision --autogenerate -m "added users table"
alembic upgrade head

Автогенерация и ограничения



Автогенерация и ограничения

Отслеживает:

- Добавление и удаление таблиц, столбцов
- Изменение типов данных
- Изменение значения nullable
- Базовые изменения в индексах и уникальных ограничениях с явным именем
- Базовые изменения внешних ключей

Не отслеживает:

- Изменение имени таблицы или столбца
- Ограничения без явного имени
- Изменение порядка столбцов
- Изменения в данных

Дата миграции

Миграции, которые изменяют данные в БД

alembic revision -m "add init data to users"

```
def upgrade() -> None:
    users_table = sa.table('users', sa.column('name', sa.String), sa.column('email', sa.String))

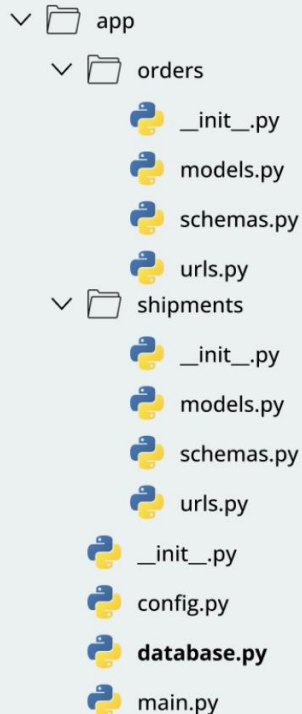
    op.bulk_insert(users_table, [{"name": "John", "email": "john@example.com"}, {"name": "Jane", "email": "jane@example.com"}])

def downgrade() -> None:
    users_table = sa.table('users', sa.column('email', sa.String))
    op.execute(users_table.delete().where(users_table.c.email.in_(["john@example.com", "jane@example.com"])))
```

alembic upgrade head

	id [PK] integer	name character varying (100)	email character varying (100)	created_at timestamp without time zone
1	1	John	john@example.com	2025-09-19 09:03:49.734137
2	2	Jane	jane@example.com	2025-09-19 09:03:49.734137

Fastapi и SQLAlchemy



```
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession
from sqlalchemy.ext.declarative import declarative_base
from typing import Annotated
from fastapi import Depends

DATABASE_URL = "postgresql+asyncpg://user:admin@localhost:5432/webdev"

engine = create_async_engine(DATABASE_URL)

AsyncSessionLocal = async_sessionmaker(bind=engine)

Base = declarative_base()

async def get_db_session() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        try:
            yield session
        except Exception as e:
            print(e)
        finally:
            await session.close()

SessionDep = Annotated[AsyncSession, Depends(get_db_session)]
```

Fastapi и SQLAlchemy

```
from ..database import Base
from sqlalchemy import Column, Integer, String

class Order(Base):
    __tablename__ = "orders"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String, index=True)
    description = Column(String, nullable=True)
    price = Column(Integer, nullable=False)
```

```
router = APIRouter()

@router.post("/", response_model=OrderRead)
async def create_order(order: OrderCreate, db: SessionDep):
    new_order = Order(**order.model_dump())
    db.add(new_order)
    await db.commit()
    # Для получения id нового заказа
    await db.refresh(new_order)

    return new_order

@router.get("/", response_model=list[OrderRead])
async def get_orders(db: SessionDep):
    orders = await db.execute(select(Order))
    return orders.scalars().all()
```

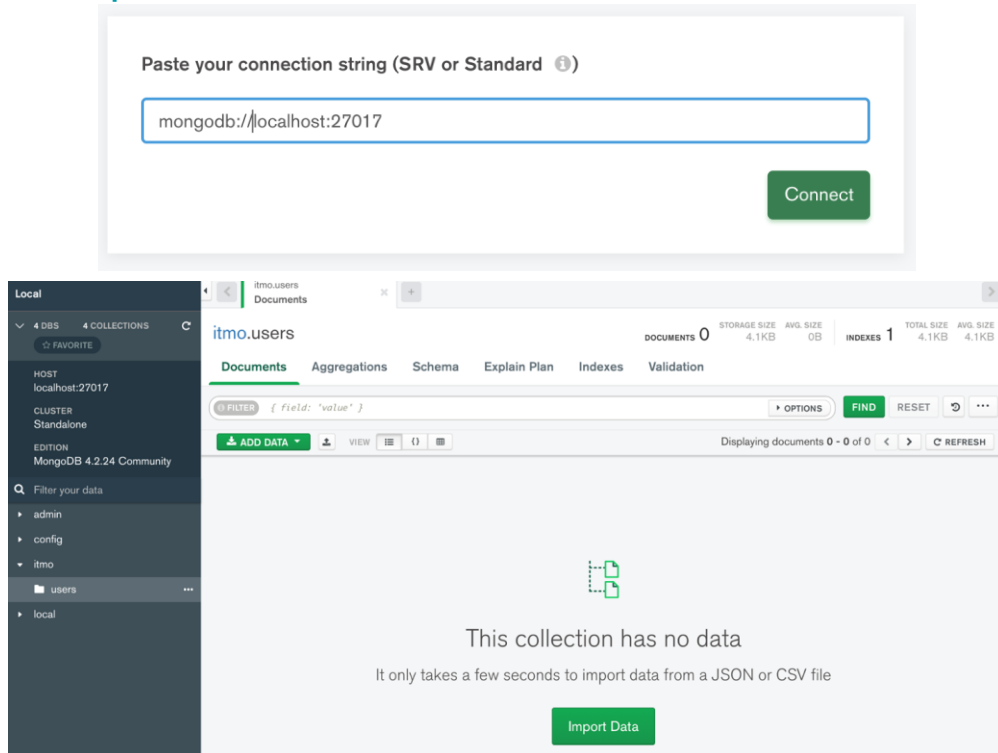
MongoDB

Запускаем БД и подключаемся к ней

<https://www.mongodb.com/products/compass>

```
services:
  mongodb:
    image: mongo:4.2
    container_name: mongodb
    ports:
      - "127.0.0.1:27017:27017"
    volumes:
      - ./data:/data/db
```

> **docker compose up -d**



Особенности MongoDB

- Rich query language (CRUD, aggregate, text search, geospatial)
- High availability (replication, automatic failover, data redundancy)
- Horizontal scalability (sharding data)
- Multiple storage engines (file, encryption, in-memory)

JSON

Типы данных:

- String
- Number (int, float)
- Object (nested JSON)
- Array
- Boolean
- null

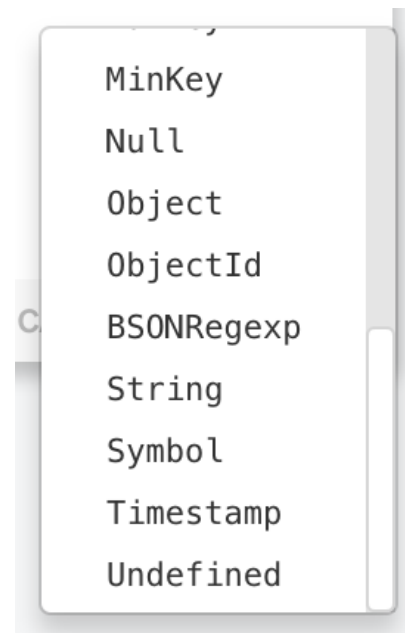
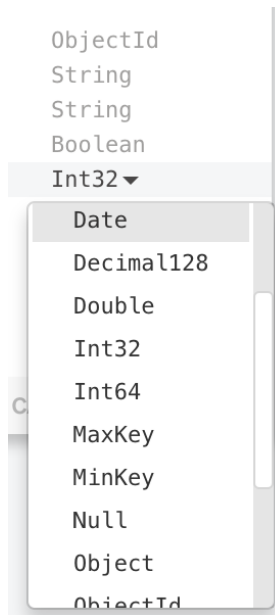
```
{
  "firstName": "John",
  "lastName": "Smith",
  "isAlive": true,
  "age": 27,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ],
  "children": [],
  "spouse": null
}
```

JSON визуализация

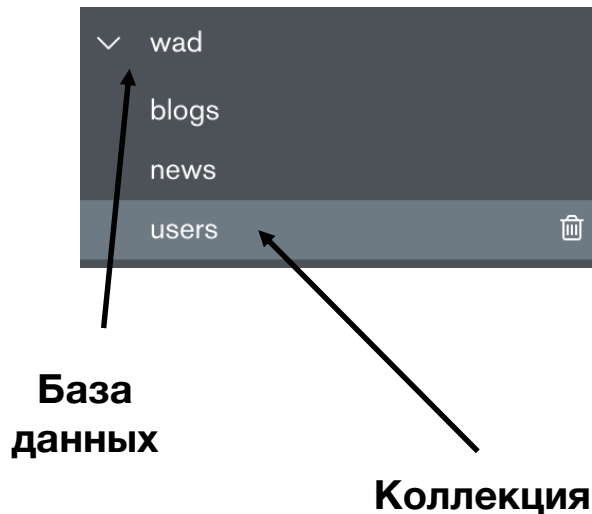
 firstName	John	String
 lastName	Smith	String
 isAlive	true	Boolean
 age	27	Int32
▼  address	{ 4 fields }	Object
 streetAddress	21 2nd Street	String
 city	New York	String
 state	NY	String
 postalCode	10021-3100	String
▼  phoneNumbers	[2 elements]	Array
▼  [0]	{ 2 fields }	Object
 type	home	String
 number	212 555-1234	String
▼  [1]	{ 2 fields }	Object
 type	office	String
 number	646 555-4567	String
▼  children	[0 elements]	Array
 spouse	null	Null

BSON

- Бинарный
- Статическая типизация
- Похож на привычный JSON



Базы данных и коллекции



Документ

```
_id: ObjectId("5e9975a7e8de390f5968e28d")
username: "user"
password: "123456"
registerDate: 2020-04-18T12:00:00.000+00:00
```

```
1  _id: ObjectId("5e9976a42482b3456c1dc4f6")
2  firstName : "John "
3  lastName  : "Smith "
4  isAlive   : true
5  age       : 27
6  > address : Object
7  > phoneNumbers : Array
8  > children  : Array
9  spouse    : null
```

Key-value

Ограничения:

- Ключ не может содержать: `null`, `$`, `.`
- Максимальная глубина вложенности: 100
- Регистр **независимые** названия баз данных (db)
- Размер документа не более **16мб**

```
_id: ObjectId("5e9976a42482b3456c1dc4f6")
firstName : "John "
lastName : "Smith "
isAlive : true
age : 27
address : Object
  streetAddress : "21 2nd Street "
  city : "New York "
  state : "NY "
  postalCode : "10021-3100 "
phoneNumbers : Array
  0 : Object
    type : "home "
    number : "212 555-1234 "
  1 : Object
    type : "office "
    number : "646 555-4567 "
children : Array
  spouse : null
```

Просмотр БД и выбор БД

```
> show dbs
```

```
admin      0.000GB
```

```
config     0.000GB
```

```
local      0.000GB
```

```
itmo       0.000GB
```

```
> use itmo
```

```
switched to db itmo
```

```
> show collections
```

```
blogs
```

```
news
```

```
users
```

Перечисление названий БД (вызываем JS методы)

```
> db.getCollectionNames().forEach(function(collname) {  
... print(collname + ": " + db[collname].count({}));  
... })
```

```
blogs: 0  
news: 0  
users: 2
```

```
> db.users.stats()  
{  
  "ns" : "itmo.users",  
  "size" : 429,  
  "count" : 2,  
  "avgObjSize" : 214,  
  "storageSize" : 36864,  
  "capped" : false,  
  ...  
}
```

Key	Value
▼ (1)	{ 13 fields }
ns	wad.users
size	429
count	2
avgObjSize	214
storageSize	36864
capped	false
▶ wiredTiger	{ 14 fields }
nindexes	1
▶ indexBuilds	[0 elements]
totalIndexSize	36864
▶ indexSizes	{ 1 field }
scaleFactor	1
ok	1.0

Вставка данных

```
> db.users.insert({"username" : "admin", "password" :  
"222", "registerDate" : ISODate("2020-04-  
18T11:00:00Z") })
```

```
WriteResult({ "nInserted" : 1 })
```

```
> db.users.insert({"username" : "guest"})
```

```
WriteResult({ "nInserted" : 1 })
```

```
_id: ObjectId("5e998e4b124338fa2eacb9b9")  
username: "admin"  
password: "222"  
registerDate: 2020-04-18T11:00:00.000+00:00
```

```
_id: ObjectId("5e998e73124338fa2eacb9ba")  
username: "guest"
```

Вставка данных различной вложенности

```
> db.news.insert(  
  {"title": "WebDev Lecture #2", "content": "Content is  
empty for now", "createDate": ISODate("2020-04-  
18T12:00:00Z"), "tags": ["education", "itmo",  
"databases"]  
})  
WriteResult({ "nInserted" : 1 })
```

```
  _id: ObjectId("66e8215921abfae98c69afa6")  
  title: "WebDev Lecture #2"  
  content: "Content is empty for now"  
  createDate: 2020-04-18T12:00:00.000+00:00  
  tags: Array  
    0: "education"  
    1: "itmo"  
    2: "databases"
```

ObjectId (mongodb: 4.2)

- Default primary key
- Generates automatically
- 12 bytes size

Size	Description
4 bytes	Seconds since Unix epoch
5 bytes	Random: Machine identifier + Process id
3 bytes	Counter starting from random value

Выборка всех данных

```
> db.users.find()
{ "_id" : ObjectId("5e9975a7e8de390f5968e28d"), "username" : "user",
  "password" : "123456", "registerDate" : ISODate("2020-04-18T12:00:00Z") }
{ "_id" : ObjectId("5e9976a42482b3456c1dc4f6"), "firstName" : "John",
  "lastName" : "Smith", "isAlive" : true, "age" : NumberLong("27000000666660"),
  "Age" : "123", "address" : { "streetAddress" : "21 2nd Street", "city" : "New
  York", "state" : "NY", "postalCode" : "10021-3100" }, "phoneNumbers" : [ {
  "type" : "home", "number" : "212 555-1234" }, { "type" : "office", "number" :
  "646 555-4567" } ], "children" : [ ], "spouse" : null }
{ "_id" : ObjectId("5e998e4b124338fa2eachb9b9"), "username" : "admin",
  "password" : "222", "registerDate" : ISODate("2020-04-18T11:00:00Z") }
{ "_id" : ObjectId("5e998e73124338fa2eachb9ba"), "username" : "guest" }
```


Фильтрация данных при выборке

```
> db.users.find({"username": "user"})
{
  "_id" : ObjectId("5e9975a7e8de390f5968e28d"),
  "username" : "user",
  "password" : "123456",
  "registerDate" : ISODate("2020-04-18T12:00:00Z")
}
> db.news.find({"tags.0": "education"})
{ "_id" : ObjectId("5e9993eb124338fa2each9c5"),
  "title" : "WAD Lecture #4", "content" : "Content is
empty for now", "createDate" : ISODate("2020-04-
18T12:00:00Z"), "tags" : [ "education", "wad",
"databases" ] }
```

<=>

```
> db.news.find({"createDate": {$gte: new  
ISODate("2012-01-12T20:15:31Z")}})  
{ "_id" : ObjectId("5e9993eb124338fa2eachb9c5"),  
  "title" : "WAD Lecture #4", "content" : "Content  
is empty for now", "createDate" : ISODate("2020-  
04-18T12:00:00Z"), "tags" : [ "education", "wad",  
"databases" ] }
```

```
> db.news.find({"createDate": {$lt: new  
ISODate("2012-01-12T20:15:31Z")}})  
>
```

Удаление

```
> db.users.deleteOne({})
```

```
WriteResult({ "nRemoved" : 4 })
```

```
> db.news.deleteOne({"tags.1": "itmo"})
```

```
WriteResult({ "nRemoved" : 1 })
```

```
> db.news.insert({"title": "ITMO Lecture #2", "content":  
"Content is empty for now", "createDate": ISODate("2020-04-  
18T12:00:00Z"), "tags": {"education":true, "itmo":true,  
"databases":true}})
```

```
WriteResult({ "nInserted" : 1 })
```

```
> db.news.deleteMany({"tags.itmo": true})
```

```
WriteResult({ "nRemoved" : 1 })
```

\$exists

```
> db.users.deleteOne({})
WriteResult({ "nRemoved" : 4 })
> db.news.deleteOne({"tags.1": "itmo"})
WriteResult({ "nRemoved" : 1 })

> db.news.insert({"title": "ITMO Lecture #2", "content": "Content is
empty for now", "createDate": ISODate("2020-04-18T12:00:00Z"),
"tags": {"education":true, "itmo":true, "databases":true}})
WriteResult({ "nInserted" : 1 })

> db.news.deleteOne({"tags.itmo": true})
WriteResult({ "nRemoved" : 1 })

> db.news.deleteOne({"tags.itmo": {$exists: true}})
WriteResult({ "nRemoved" : 1 })
```

Модификация

```
> db.news.updateMany( {"tags.itmo": true}, {$set:
{"tags.python": true}})
{ "acknowledged" : true, "matchedCount" : 2,
"modifiedCount" : 2 }
```

```
_id: ObjectId("66e8285d21abfae98c69afba")
title: "ITMO Lecture #2"
content: "Content is empty for now"
createDate: 2020-04-18T12:00:00.000+00:00
✓ tags: Object
  education: true
  itmo: true
  databases: true
```

```
_id: ObjectId("66e8285d21abfae98c69afba")
title: "ITMO Lecture #2"
content: "Content is empty for now"
createDate: 2020-04-18T12:00:00.000+00:00
✓ tags: Object
  education: true
  itmo: true
  databases: true
  python: true
```

Upsert

```
> db.users.update({"username":"user"}, {$set: {"password":  
"newpass"}}, {upsert: true})  
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1  
})  
> db.users.update({"username":"guest"}, {$set: {"password":  
"newpass"}}, {upsert: true})  
WriteResult({  
  "nMatched" : 0,  
  "nUpserted" : 1,  
  "nModified" : 0,  
  "_id" : ObjectId("5e99a4ae2482b3456c1dd2cc")  
})
```

▼ (1) ObjectId("5e99a2cb124338fa2eacb9cf")	{ 3 fields }
_id	ObjectId("5e99a2cb124338fa2eacb9cf")
username	user
password	user
▼ (2) ObjectId("5e99a2d1124338fa2eacb9d0")	{ 3 fields }
_id	ObjectId("5e99a2d1124338fa2eacb9d0")
username	admin
password	admin

Limit, Skip

```
> db.users.find({}).limit(3)
```

```
5e99a2cb124338fa2eachb9cf
```

```
5e99a2d1124338fa2eachb9d0
```

```
5e99a4ae2482b3456c1dd2cc
```

```
> db.users.find({}).limit(1).skip(0)
```

```
{ "_id" : ObjectId("5e99a2cb124338fa2eachb9cf"),  
  "username" : "user", "password" : "newpass" }
```

```
> db.users.find({}).limit(1).skip(1)
```

```
... 5e99a2d1124338fa2eachb9d0 ...
```

```
> db.users.find({}).limit(1).skip(2)
```

```
5e99a4ae2482b3456c1dd2cc
```

Sort

```
> db.users.find({}).sort({"username":1})  
admin, guest, user
```



```
> db.users.find({}).sort({"username":-1})  
user, guest, admin
```



Two fields:

```
> db.users.find({}).sort({"age": -1, "username":  
-1})
```


Import and Export

```
mongodump --host "localhost" --port 27017 -d itmo  
-o ./output
```

```
mongorestore --host "localhost" --port 27017 -d  
itmo ./output
```

MongoDB & FastAPI

MongoEngine

<http://mongoengine.org/>

```
import mongoengine as me
from pydantic import BaseModel

# MongoDB connection setup
DATABASE_URL = "mongodb://localhost:27017/itmo"
me.connect(host=DATABASE_URL)
```

Легче легкого

Шаг 1 – пишем UserResponse

Шаг 2 – пишем класс User

Шаг 3 – пишем main.py

```
class UserResponse(BaseModel):  
    id: str  
    email: str  
  
class User(me.Document):  
    email = me.StringField(required=True,  
unique=True)  
    hashed_password =  
me.StringField(required=True)  
  
    meta = {'collection': 'users'}
```

```
from fastapi import FastAPI, Depends, HTTPException  
from database import User, UserResponse  
  
app = FastAPI()  
  
@app.post("/users/", response_model=UserResponse)  
async def create_user(email: str, password: str):  
    if User.objects(email=email).first():  
        raise HTTPException(status_code=400, detail="Email already registered")  
  
    user = User(email=email, hashed_password=password)  
    user.save()  
  
    return UserResponse(id=str(user.id), email=user.email)  
  
@app.get("/users/{email}", response_model=UserResponse)  
async def get_user(email: str):  
    user = User.objects(email=email).first()  
    if not user:  
        raise HTTPException(status_code=404, detail="User not found")  
    return UserResponse(id=str(user.id), email=user.email)
```

Building large applications

Зачем?

- Приложения масштабируются
- Появляется версионирование
- Большая команда разработки
- Необходимость тестирования

<https://github.com/Netflix/dispatch/tree/master>

MVC (Model-View-Controller)

Model – представляет данные и бизнес-логику приложения.

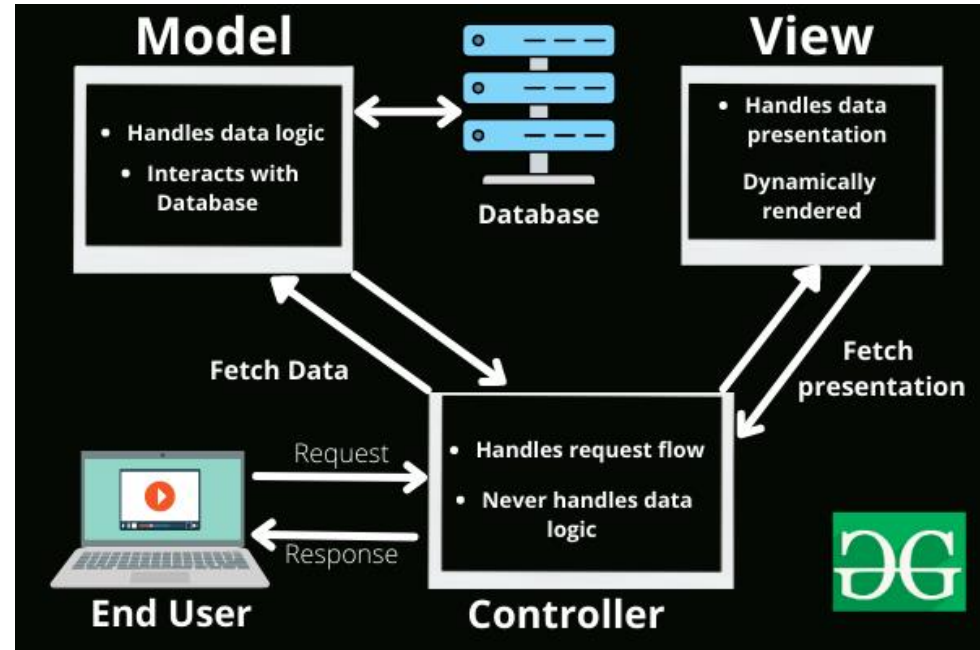
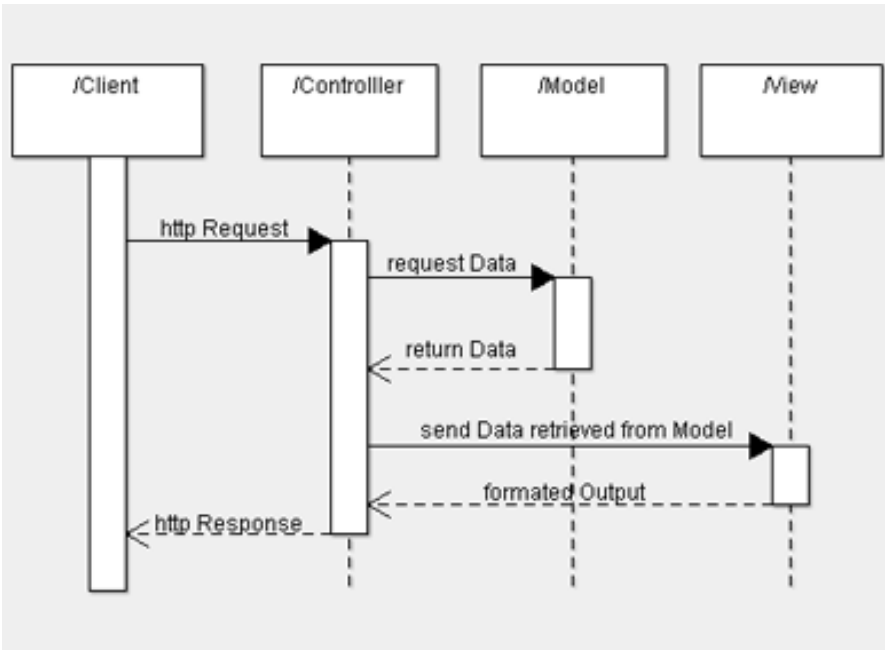
- получение данных из базы данных
- манипуляция данными (бизнес логика)

View – часть пользовательского интерфейса (UI) приложения.

- отображение данных, предоставленных Моделью

Controller – посредник между моделью и представлением. Обрабатывает запросы пользователя, взаимодействует с Моделью для получения или обновления данных и решает, какое представление должно быть отображено.

MVC



MCS (Model-Controller-Service)

Model – модель по-прежнему представляет данные и бизнес-логику приложения.

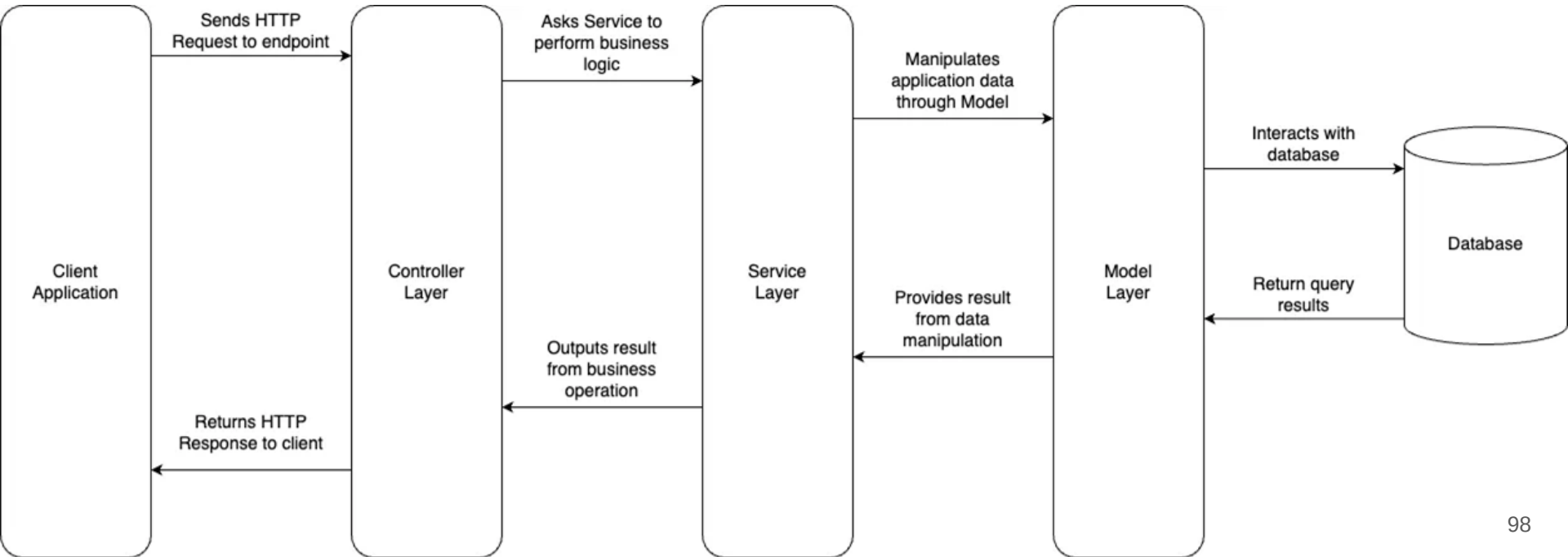
Controller – контроллер по-прежнему отвечает за обработку запросов, но его основная задача теперь заключается в делегировании сложной бизнес-логики слою **Service**.

Service – инкапсуляция бизнес-логики, которая в противном случае загромождала бы Controller. Службы решают задачи:

- получение данных
- выполнение сложных операций

Контроллеры теперь более компактные и содержат только маршрутизацию и обработку запросов.

MCS (Model-Controller-Service)



Вопросы?